

Prvi izpitni rok pri predmetu Programiranje 1

29. januar 2025

Dopolnite in oddajte datoteke `Prva.java`, `Druga.java`, `Tretja.java` in `Cetrta.java`. Testirate jih lahko takole:

(1) `tj.exe Prva.java . .` (2) `tj.exe Druga.java . .` (3) `tj.exe` (4) `tj.exe`

- ① V trgovini izvajajo posebno akcijo: na vsakih k kupljenih izdelkov je najcenejši med njimi zastonj. Napišite program, ki prebere število $k \in [1, 10^4]$ (prva vrstica vhoda) in zaporedje $n \in [1, 10^4]$ cen izdelkov z intervala $[1, 10^4]$ (druga vrstica vhoda), izpiše pa skupni znesek plačila. Število n ni znano vnaprej (cene berite do konca vhoda).

V 50% testnih primerov velja $k = n$.

test01.in:

3
2 3 1 6 6 7 8 3 4 5 6

test01.out:

41

Prva trojica izdelkov (2, 3, 1) nas stane 5, za drugo (6, 6, 7) plačamo 13, tretja (8, 3, 4) pa znese 12. Na koncu plačamo še 11 za zadnja dva izdelka ($2 < k$).

- ② Zaporedoma bomo privili n vijakov, ki pripadajo tipom $0, 1, \dots, m-1$ (npr. 0 = križni, 1 = ploščati, 2 = imbus ...). Za vsakega od m tipov imamo po k vijačnikov. Vijak privijemo v t_v časovnih enotah, vijačnik pa se mora nato hladiti še t_h časovnih enot, preden je na voljo za naslednje vijačenje. Napišite program, ki izpiše porabo časa do trenutka, ko privijemo zadnji vijak.

V prvi vrstici vhoda so zapisana števila n, m, k, t_v in t_h (vsa pripadajo intervalu $[1, 100]$), druga pa vsebuje n števil z intervala $[0, m-1]$ (tipi posameznih vijakov).

V 50% testnih primerov velja $k = 1$.

test01.in:

8 2 2 1 5
1 1 1 0 0 0 0 1

test01.out:

16

Za lažjo predstavo bomo vijačnike poimenovali A in B (za tip 0) ter C in D (za tip 1). Naslednja tabela prikazuje potek vijačenja:

Vijak	Čas zač. → vijačnik → čas konca	Vijačniki se ohladijo ob času ...
		$A \mapsto 0, B \mapsto 0, C \mapsto 0, D \mapsto 0$
1 (tip 1)	$0 \rightarrow C \rightarrow 1$	$A \mapsto 0, B \mapsto 0, C \mapsto 6, D \mapsto 0$
2 (tip 1)	$1 \rightarrow D \rightarrow 2$	$A \mapsto 0, B \mapsto 0, C \mapsto 6, D \mapsto 7$
3 (tip 1)	$6 \rightarrow C \rightarrow 7$	$A \mapsto 0, B \mapsto 0, C \mapsto 12, D \mapsto 7$
4 (tip 0)	$7 \rightarrow A \rightarrow 8$	$A \mapsto 13, B \mapsto 0, C \mapsto 12, D \mapsto 7$
5 (tip 0)	$8 \rightarrow B \rightarrow 9$	$A \mapsto 13, B \mapsto 14, C \mapsto 12, D \mapsto 7$
6 (tip 0)	$13 \rightarrow A \rightarrow 14$	$A \mapsto 19, B \mapsto 14, C \mapsto 12, D \mapsto 7$
7 (tip 0)	$14 \rightarrow B \rightarrow 15$	$A \mapsto 19, B \mapsto 20, C \mapsto 12, D \mapsto 7$
8 (tip 1)	$15 \rightarrow D \rightarrow 16$	$A \mapsto 19, B \mapsto 20, C \mapsto 12, D \mapsto 21$

Prvega vijaka se ob času 0 lotimo z vijačnikom C in ga privijemo ob času 1. Vijačnik C se bo hladil do časa 6. Drugega vijaka se ob času 1 lotimo z vijačnikom D in ga privijemo ob času 2. Vijačnik D se bo hladil do časa 7. Tretjega vijaka se lotimo šele ob času 6, ko se ohladi vijačnik C .

- ③ Vrt je razdeljen na grede, na vsaki gredi pa gojimo zaporedje zaplat določenih vrtnin. Objekt `grede[i][j]` predstavlja j -to zaplato na i -ti gredi. Na primer, če na gredi z indeksom 0 gojimo zaplato graha, zaplato čebule, še eno zaplato graha, zaplato korenja in zaplato fižola, potem sta `grede[0][0]` in `grede[0][2]` dva različna objekta tipa `Grah`, `grede[0][1]` je objekt tipa `Cebula` itd.

V razredu `Vrt` dopolnite sledeči metodi:

- [50%] `public boolean monokultura(int ixGrede)`

Vrne `true` natanko v primeru, če vse vrtnine na gredi s podanim indeksom pripadajo istemu razredu (npr. če na gredi gojimo samo zaplate graha). Pomagajte si z metodo `imeRazreda`, ki vrne ime razreda, ki mu pripada objekt, na katerega kaže kazalec `this`. Na primer:

```
Vrtnina v = new Grah();
System.out.println(v.imeRazreda()); // Grah
```

Če greda vsebuje kvečjemu eno zaplato, naj metoda vrne `true`.

- [50%] `public int meraSosednosti()`

Vrne vsoto mer sosednosti za posamezne grede, pri čemer je mera sosednosti za določeno gredo enaka vsoti mer sosednosti za posamezne pare sosednjih vrtnin na tej gredi, mere sosednosti za posamezne pare vrtnin pa so podane v tabeli `SOSEDNOST`. Mera sosednosti za gredo s kvečjemu eno zaplato je enaka 0.

Na primer, pri gredi (grah, čebula, grah, korenje, fižol) imamo štiri pare sosed: grah-čebula (mera = -1), čebula-grah (mera = -1), grah-korenje (mera = 1) in korenje-fižol (mera = 0). Mera sosednosti za takšno gredo je torej $-1 + (-1) + 1 + 0 = -1$.

Namig: poleg tabele `SOSEDNOST` vam bo morda prišla prav tudi tabela `RAZREDI`.

- ④ Dopolnite metodo

```
public static Iterator<String> piramidator(int visina, int odmik),
```

ki vrne iterator, s katerim je mogoče narisati piramidoTM s podano višino ($1 \leq \text{visina} \leq 200$) in odmikom od levega roba ($0 \leq \text{odmik} \leq 200$), pri čemer namesto presledkov uporabljamo minuse. Če boste iterator pravilno napisali, bo koda ...

```
Iterator<String> it = Cetrti.piramidator(5, 3);
while (it.hasNext()) {
    System.out.print(it.next());
}
```

... proizvedla sledeči izpis:

```
-----*
-----***
-----*****
-----*****
-----*****
```

Iteratorjeva metoda `next` naj vsakokrat vrne enega od nizov "-", "*" in `String.format("%n")` (prelom vrstice). Seveda se mora tudi zadnja vrstica končati z nizom `String.format("%n")`.

V 50% testnih primerov velja `visina = 1`.